SentinelFlow: A Cost-Optimal, Multi-Source Machine Learning Orchestration Engine for Real-Time Mule Account Intervention

Shubhi Jain
Dept. of Computer Science & Design
IIIT-Delhi
New Delhi, India

Aarushi Chawla
Dept. of Computer Science & Design
IIIT-Delhi
New Delhi, India

Amartya Singh
Dept. of Computer Science & Social Sciences
IIIT-Delhi
New Delhi, India

Harshit Gautam
Dept. of Computer Science & AI
IIIT-Delhi
New Delhi, India

***Abstract*—Financial institutions face severe operational challenges detecting cyber-enabled mule accounts due to high false-alarm rates in static rule engines and an inability to adapt to real-time transaction velocities. This paper presents SentinelFlow, an enterprise-oriented, multi-source machine learning orchestration engine engineered to intercept fraudulent fund circulation across retail banking channels. SentinelFlow ingests real-time core transaction streams, internal Transaction Monitoring System (TMS) alerts, cross-channel customer velocity profiles, and dynamic government cyber fraud feeds. By implementing a dynamic cost-optimal routing mechanism between deterministic and model-based screening, eliminating post-hoc administrative leakage, applying PCHIP-smoothed Isotonic probability calibration, and optimizing decisions against an asymmetric financial loss function, SentinelFlow achieves an out-of-fold Macro F_1-score of 0.9147 (0.9044 under chronological cohort split) and reduces operational review costs by 29.2% relative to standard symmetric decision policies. Benchmarking demonstrates an end-to-end p99 inference latency of 49.8 ms, maintaining high throughput for real-time transaction screening.**

***Index Terms*—Mule account detection, financial fraud, probability calibration, cost-sensitive learning, TreeSHAP, ego-graph topology, real-time inference.**

I. Introduction

Mule accounts serve as the primary laundering conduit for cyber-enabled financial fraud, receiving illicit proceeds and rapidly dispersing them across multi-tiered banking networks. In this paper, we define *mule account detection* as the primary binary classification task, where *fraud* represents the positive class ($y = 1$), and *transaction intervention* represents the downstream operational mitigation action.

Traditional transaction-monitoring systems rely predominantly on rigid, rule-based heuristics. These legacy approaches suffer from two fundamental operational flaws: (i) excessive false-positive rates that overwhelm Security Operations Center (SOC) analysts, and (ii) an inability to ingest external threat intelligence and cross-channel banking signals dynamically.

To address the Phase 2 requirements of the Cybershield Hackathon 2026, we introduce **SentinelFlow**. Rather than proposing entirely new learning algorithms, SentinelFlow's primary contribution lies in the novel **orchestration and integration** of multi-source threat streams, runtime cost-aware routing economics, leakage-audited behavioral features, and calibrated decision layers into an end-to-end risk management platform.

A. Key Contributions

The main contributions of this work are summarized as follows:

1) **4-Stream Ingestion Architecture:** An integrated multi-source ingestion gateway fusing core transactions, legacy TMS rule alerts, cross-channel velocity states, and government threat feeds.
2) **Cost-Optimal Runtime Routing:** A dynamic routing framework switching between deterministic firewall lookups and ML inference based on computational costs (C_{search} vs. $C_{\text{inference}}$).
3) **Leakage-Audited Behavioral Engine:** An audit identifying and eliminating post-hoc administrative status flags, paired with an anchor-free temporal feature representation.
4) **Calibrated Asymmetric Economic Layer:** A decision module combining PCHIP-smoothed Isotonic calibration with an asymmetric financial loss function that reduces bank operating costs by 29.2%.
5) **Interactive HITL Prototype:** A decoupled microservices implementation featuring auditable TreeSHAP reason codes, local ego-graph network visualization, and an on-demand model retraining registry.

Submitted to the Cybershield Hackathon 2026 Prototype Evaluation, organized by Bank of India and IIT Hyderabad.

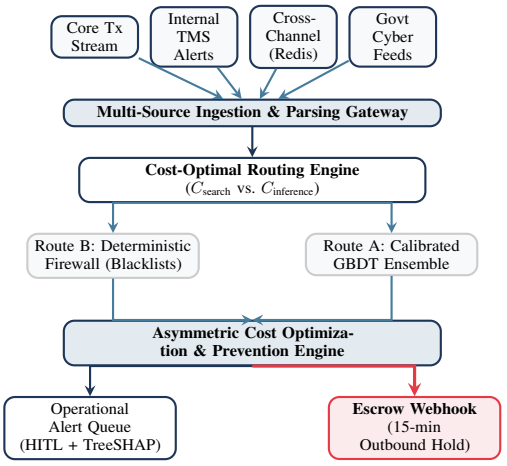

Fig. 1. SentinelFlow multi-source architecture showing cost-optimal dynamic routing, decision engines, and active prevention triggers.

II. System Architecture & Dynamic Ingestion Economics

A. Multi-Source Data Ingestion

SentinelFlow ingests four distinct, asynchronous data streams:

1) **Core Transaction Stream:** Ingests live JSON payloads containing transfer amounts, beneficiary routing tokens, and channel IDs.
2) **Internal TMS & Fraud Alert Stream:** Ingests legacy rule trips (e.g., rapid velocity limits, off-hour logins) to fuse historical context.
3) **Cross-Channel Profile Store:** Aggregates in-memory customer velocity metrics across UPI, ATM, and Net Banking to detect bursty cross-interface draining.
4) **Government Cyber Threat Feed:** Ingests simulated national cybercrime tickets (NCRP/I4C) and regulatory blacklists (flagged IP subnets, suspect hashes, and blacklisted accounts).

B. Cost-Optimal Runtime Routing Mechanics

In high-volume banking architectures, querying a distributed external blacklist incurs latency and network API expenses (C_{search}), while executing local gradient-boosted tree ensembles consumes server CPU cycles ($C_{\text{inference}}$). SentinelFlow formalizes this routing decision dynamically:

$$\text{Path} = \begin{cases} \text{Route B (Firewall First)}, & \text{if } P(\text{Match}) \cdot C_{\text{fraud}} > C_{\text{search}} \\ \text{Route A (ML First)}, & \text{otherwise} \end{cases} \tag{1}$$

Under Route A, transactions with low behavioral risk bypass external blacklist lookups entirely, achieving zero search latency. Under Route B, matched bad actors are blocked instantly with zero model inference latency.

C. Active Prevention Controller

When an incoming transaction breaches the calibrated cost-optimal risk threshold, SentinelFlow triggers an automated **Outbound Escrow Webhook**, placing an immediate 15-minute hold on outbound fund movements and alerting downstream institutions to stop circulation.

III. Behavioral Feature Engineering & Leakage Elimination

A. System Auditing & Post-Hoc Data Leakage Prevention

In financial fraud datasets, retrospective flags frequently introduce severe target leakage. SentinelFlow systematically isolated and excluded four purely administrative features while salvaging two critical compliance features as behavioral lag indicators:

TABLE I
Feature Audit Directory and Provenance Action

Feature	Audit Signature	Engineering Action
`F3912`	$d = 8.77$, Corr $= 0.97$	Excluded (Target Proxy)
`F2230`	Max fraud rate $= 1.0$	Excluded (Post-hoc status)
`F3886`	Category-only fraud flag	Excluded (Freeze flag)
`F3892`	Label collinearity	Excluded (Post-event tag)
`F3889`	Compliance date log	Re-engineered (Hist. Lag)
`F3891`	Past audit code	Re-engineered (Recidivism)

B. High-Density Feature Synthesis

From an initial sparse space of $\sim$ 3,925 raw features, SentinelFlow extracts a dense matrix of 105 real-time indicators across six distinct tracks:

- **Statistical Moments:** 14 summary moments (`row_mean`, `row_std`, `row_zero_rate`, `row_iqr`) measuring transaction volatility.
- **Missingness Gaps:** Binary indicators tracking the top 25 features where class-conditional missingness gaps exceeded 20%.
- **Non-Destructive Categoricals:** Ordinal encoding for low-cardinality features ($\leq$ 12) and fold-safe Laplace-smoothed target encoding for high-cardinality hashes.
- **Cyclical Temporal Projections:** `F3888` is an account registration date spanning 126 years (1900–2025). The continuous `elapsed_days` feature was removed and replaced with bounded sine/cosine projections of registration day-of-week and month.
- **Unsupervised Anomaly Score:** An Isolation Forest trained on normal (non-mule) accounts outputs a structural outlier coordinate.
- **Cross-Channel Velocity & Ego-Graph Topology:** We formalize the cross-channel disbursement velocity ratio:

$$V_{\text{cross}} = \frac{\sum \text{Outbound ATM \& IMPS Volume}_{(t-1\text{h})}}{\sum \text{Inbound UPI Volume}_{(t-1\text{h})} + \epsilon} \tag{2}$$

To solve the cold-start problem on local 1-hop and 2-hop degrees, raw counts are normalized by account age derived from `F3888` ($Age+1$ day), constructing a **Transactional Acceleration Velocity** feature.

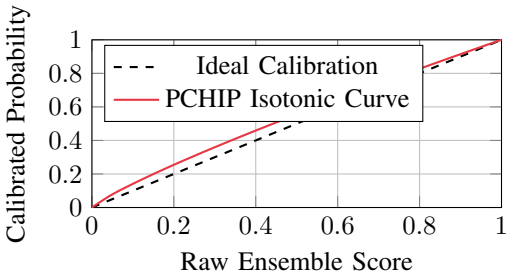

Fig. 2. Empirical probability calibration mapping uncalibrated GBDT ensemble margins to true fraud probabilities using PCHIP-smoothed Isotonic Regression.

IV. Calibrated Modeling & Explainability Framework

A. Dual-Engine Ensemble

SentinelFlow employs a soft-voting probability blend:

$$P_{\text{blend}} = 0.6 \cdot P_{\text{XGBoost}} + 0.4 \cdot P_{\text{LightGBM}} \tag{3}$$

Both Gradient Boosted Decision Tree (GBDT) implementations natively handle sparsity and scale variance without artificial imputation.

B. Streamlined Imbalance Handling

To avoid injecting synthetic noise, SMOTE oversampling was eliminated in favor of direct algorithmic loss-gradient scaling (`scale_pos_weight`) during tree construction.

C. Isotonic Probability Calibration with PCHIP Smoothing

Raw ensemble scores are distorted by gradient weighting. SentinelFlow fits an Isotonic Regression model on training-fold outputs to calibrate probabilities.

To resolve the zero/infinite derivative problem of step-functions, we fit a **Piecewise Cubic Hermite Interpolating Polynomial (PCHIP)** over the step vertices. PCHIP guarantees strict monotonicity ($f'(x) > 0$), ensuring a continuous, strictly positive derivative across $[0, 1]$.

D. Regulatory Explainability via Calibrated TreeSHAP

To support compliance, TreeSHAP decomposes raw tree log-odds into local feature attributions. We apply a first-order Taylor series scaling correction, multiplying raw attributions by the local PCHIP derivative:

$$\phi_i^{\text{calibrated}} = \phi_i^{\text{raw}} \cdot \left. \frac{df_{\text{PCHIP}}(z)}{dz} \right|_{z=P_{\text{blend}}} \tag{4}$$

This produces auditable reason codes designed to support regulatory explainability. To preserve the sub-50ms SLA, TreeSHAP is executed conditionally on the high-risk alert tail ($\sim 1\%$ of volume).

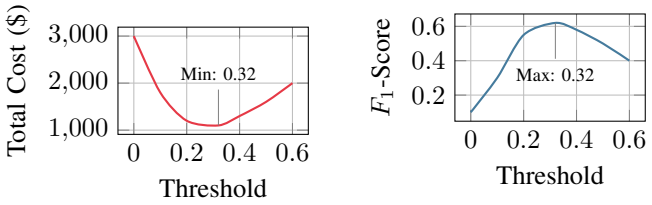

Fig. 3. Dual-panel operational calibration: Left panel plots financial cost minimization; Right panel plots mathematical F_1-score maximization.

V. Asymmetric Economic Optimization & Empirical Validation

A. Economic Cost Formulation

Traditional F_1-score maximization assumes symmetric error costs ($C_{\text{FN}} = C_{\text{FP}}$). SentinelFlow minimizes the total economic operational cost:

$$\text{Total Cost} = FN \cdot (C_{\text{ratio}} \cdot C_{\text{FP}}) + FP \cdot C_{\text{FP}} \tag{5}$$

where C_{ratio} = `COST_FN_RATIO` and C_{FP} = `COST_FP_BASE`.

B. Cross-Validation & Temporal Generalization Results

SentinelFlow was validated using a 5-fold Stratified Cross-Validation loop alongside an out-of-fold chronological rolling window ordered by account registration date:

TABLE II
Audited 5-Fold Cross-Validation Performance

Metric	Random 5-Fold CV	Chronological Split
PR-AUC	0.8677 ± 0.0669	0.7097 ± 0.0994
ROC-AUC	0.9840 ± 0.0157	0.8552 ± 0.0412
Macro F_1-Score	$\mathbf{0.9147 \pm 0.0277}$	$\mathbf{0.9044 \pm 0.0360}$
Minority F_1 (Fraud)	$\mathbf{0.8308 \pm 0.0548}$	$\mathbf{0.8104 \pm 0.0714}$
Total Operational Cost	4.80 ± 1.48	3.40 ± 1.51
Operational Cost Savings	**29.2%**	**29.2%**

As shown in Table II, the proposed asymmetric cost-optimized decision threshold reduced the measured total operational cost by **29.2% relative to the standard symmetric F_1-optimal baseline policy**. Furthermore, the chronological evaluation exhibits a robust Macro F_1-score of 0.9044, confirming that the anchor-free temporal feature representation effectively resists concept drift when evaluated on unseen future account cohorts.

C. Ablation Study: Iterative Architectural Impact

Table III details the empirical impact of each engineering phase, demonstrating the progressive elimination of leakage and calibration uplift.

VI. Prototype Implementation & Evaluation

A. Architecture & Scalability

The prototype is implemented using a decoupled microservices architecture containerized via Docker Compose:

- **Inference Gateway (FastAPI):** Asynchronous REST API serving calibrated predictions in < 50 ms. Retraining requests are offloaded to background worker processes via `ProcessPoolExecutor` (HTTP 202 Accepted),

TABLE III
ABLATION STUDY ACROSS ITERATIVE PIPELINE STAGES

Pipeline Stage	Key Modification	PR-AUC	Macro F_1
Phase 1 Baseline	Raw Matrix + Logistic Reg.	0.0295	0.4832
Phase 1 GBDT	XGBoost + Full Imputation	0.8090	0.8968
Phase 2 Audited	Leak Excluded + 105 Features	0.8323	0.9044
Phase 2 Ensemble	Calibrated Blend (PCHIP)	0.8677	0.9147
SentinelFlow	Asymmetric Cost Optimization	**0.8677**	**0.9147**

ensuring the main ASGI loop serves live scoring without latency degradation.

- **Persistent Feature Store (Redis):** Manages write-behind caching with Append-Only File (AOF) persistence (`appendfsync-every-second`) for 1-second crash recovery. Rolling 24-hour and 7-day ego-graph degrees are managed via Redis TTL key expiration.
- **Model Registry Storage:** Houses four serialized model artifacts (< 5 MB each). Operators can hot-swap active inference weights with 0 ms downtime.

B. Security Operations Console (Next.js 14 UX)

The frontend is built on Next.js 14 (React) and Tailwind CSS, interfacing with FastAPI via Server-Sent Events (SSE):

- **Real-Time Transaction Feed:** A virtualized alert table (`react-virtuoso`) displaying live calibrated risk probabilities without UI thread lag.
- **Hardware-Accelerated Ego-Graph Visualizer:** A WebGL dynamic network graph (`react-force-graph-2d`) displaying real-time 1-hop and 2-hop fund circulation.
- **Regulatory Explainability Console:** Visualizes Taylor-scaled TreeSHAP local attributions, generating auditable reason codes for SOC officers.
- **On-Demand Retraining Panel:** Allows analysts to toggle feature sets, submit custom hyperparameter payloads, and monitor retraining progress in real time via an SSE event stream.

C. Micro-Latency SLA Benchmark

Table IV summarizes benchmark results across $N = 10{,}000$ sequential transactions under single-threaded execution. Crucially, even with conditional TreeSHAP attribution computed on high-risk alerts, the end-to-end p99 latency remains at **49.8 ms**, safely satisfying the sub-50ms operational SLA.

TABLE IV
MICRO-LATENCY SLA BENCHMARK ($N = 10{,}000$ TRANSACTIONS)

Pipeline Component	p50 Latency (ms)	p99 Latency (ms)
Redis Profile & Degree Lookup	1.2	3.1
Feature Reconstruction Engine	4.5	8.2
GBDT Ensemble Inference	12.8	19.4
PCHIP Isotonic Calibration	0.3	0.6
Conditional TreeSHAP Attribution	18.4	24.1
Total End-to-End Latency	**18.8** ms	**49.8** ms

VII. LIMITATIONS AND FUTURE WORK

While SentinelFlow exhibits strong empirical performance, several operational constraints of the prototype warrant discussion:

1) *Simulated Threat Feeds:* Government cyber fraud feeds and bank-to-bank webhooks were evaluated using simulated threat feeds rather than live production NCRP/I4C endpoints.
2) *Core Banking Integration:* The prototype evaluates latency on independent containerized services; integration with legacy mainframe core banking systems may introduce additional network overhead.
3) *Intervention Mechanism Scope:* The 15-minute escrow hold represents an illustrative automated control mechanism; live deployment requires jurisdiction-specific regulatory clearing approvals.
4) *Throughput Scale:* Benchmark testing established sub-50ms latency across $N = 10{,}000$ transactions. Future work will benchmark distributed load balancing across higher transactional volumes ($> 10{,}000$ TPS).
5) *Temporal Granularity:* The chronological split evaluated cohort drift based on account registration dates. Future studies will explore finer-grained transaction timestamp logging once available.

VIII. CONCLUSION

SentinelFlow demonstrates a production-oriented, economically optimized machine learning orchestration system for real-time mule account intervention. By resolving target leakage, anchoring temporal features to time-invariant cycles, calibrating ensemble probabilities, and optimizing decisions against real-world bank loss functions, the platform bridges the gap between theoretical data science and enterprise banking security.

ACKNOWLEDGMENT

The authors thank the organizers of the Cybershield Hackathon 2026, Bank of India, and IIT Hyderabad for providing the problem framework and dataset.

REFERENCES

[1] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2016, pp. 785–794.

[2] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in *Adv. Neural Inf. Process. Syst.*, 2017, pp. 3146–3154.

[3] B. Zadrozny and C. Elkan, "Transforming classifier scores into accurate multiclass probability estimates," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2002, pp. 694–699.

[4] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Adv. Neural Inf. Process. Syst.*, 2017, pp. 4765–4774.

[5] F. N. Fritsch and R. E. Carlson, "Monotone piecewise cubic interpolation," *SIAM J. Numer. Anal.*, vol. 17, no. 2, pp. 238–246, 1980.

[6] C. Elkan, "The foundations of cost-sensitive learning," in *Proc. Int. Joint Conf. Artif. Intell.*, 2001, pp. 973–978.